Fundamentals of Programming using Python • First Year, FinTech

Chapter 05: Lists

Making and reading • slicing • mutability and the aliasing trap • copying • adding and removing • `sort` vs `sorted`

Chapter 05 of 10

Dr. Brahim Zirari

Yahia Farès University of Medea

in linkedin.com/in/brahimzirari · Google Scholar

October 2026

1. Why a list

❑ **The problem** – Six branches reported their monthly revenue. With what you know so far that is six names — `rev1` to `rev6` — and every operation written out six times. Adding a seventh branch means editing the program.

❑ **The answer** – A `list` holds **many values under one name**, in order. One name, any number of values, and the same code whether there are six branches or six hundred.

```
sales = [184500, 92300, 156800, 73400, 210600, 45900]
print(sales)
print(len(sales))
```

Output

```
[184500, 92300, 156800, 73400, 210600, 45900]
6
```

❑ **Writing one** – Square brackets, items separated by commas. A list may hold any types, even mixed, and may be empty.

```
branches = ["Medea", "Alger", "Oran"]
mixed = ["Medea", 184500, 0.035, True]
empty = []
print(len(branches), len(mixed), len(empty))
```

Output

```
3 4 0
```

Remark: mixing types is allowed but rarely wise. A list whose items are all the same kind of thing — all revenues, all branch names — can be summed, sorted and compared. A mixed list cannot, and you will discover this at the moment you try.

2. Reading a list

❑ **Same rules as strings** – Positions start at **0**; negative indices count from the end; a slice takes from `a` up to but *not including* `b`.

```
sales = [184500, 92300, 156800, 73400, 210600, 45900]
print(sales[0], sales[2], sales[-1])
print(sales[0:3])
print(sales[3:])
print(sales[-2:])
print(sales[::-1])
```

Output

```
184500 156800 45900
[184500, 92300, 156800]
[73400, 210600, 45900]
[210600, 45900]
[45900, 210600, 73400, 156800, 92300, 184500]
```

❑ **A slice of a list is a list** – `sales[0]` is a number; `sales[0:1]` is a *list* containing one number. The brackets in the output tell you which you have.

Pitfall: `sales[6]` on a six-item list is an `IndexError`: the last position is 5. But `sales[6:]` is not an error — it quietly returns `[]`. As with strings, an index complains and a slice does not.

3. Lists can be changed

❑ **The difference that matters** – A string is **immutable**: every method returns a new one. A list is **mutable**: you can change it in place, and its name keeps pointing at the same list.

```
sales = [184500, 92300, 156800]
sales[1] = 99000
print(sales)

name = "Medea"
# name[0] = "X"   <- TypeError: strings cannot be changed
```

Output

```
[184500, 99000, 156800]
```

❑ **Slice assignment** – A whole slice can be replaced at once, and the replacement need not be the same length.

```
sales = [184500, 92300, 156800, 73400]
sales[0:2] = [200000]
print(sales)
```

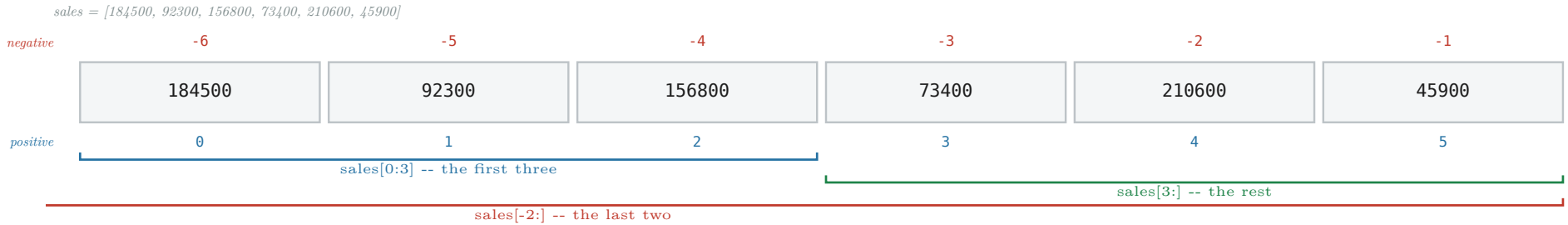


Figure 1. Six branch revenues, two numbering systems, three slices.

Output

```
[200000, 156800, 73400]
```

4. The aliasing trap

❑ **Assignment still does not copy** – Chapter 03 said that a name is a label attached to a value. For numbers and strings that never caused trouble, because those values cannot change. A list *can*, and now it matters.

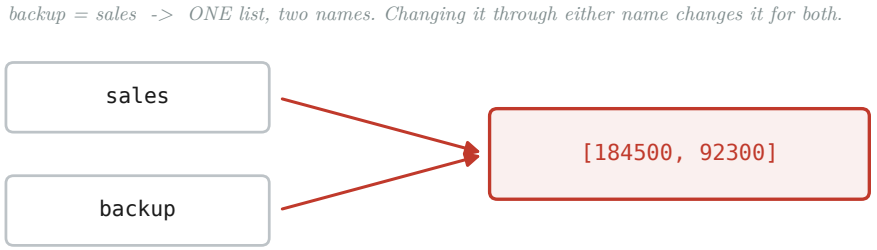


Figure 2. After `backup = sales` there is one list and two labels on it.

Pitfall: ***this produces no error.*** *A "backup" made this way is not a backup: it is a second label on the original, and every change reaches it. In a program that cleans data, this quietly destroys the thing you kept in order to compare against.*

5. Making a real copy

❑ **Three ways, all equivalent** – A slice of everything, the `list` function, or `.copy()`. Each builds a new list.

```
sales = [184500, 92300]

a = sales[:]
b = list(sales)
c = sales.copy()

sales.append(500)
print(sales)
print(a, b, c)
```



Figure 3. The consequence. `backup` was never touched, and changed anyway.

Output

```
[184500, 92300, 500]
[184500, 92300] [184500, 92300] [184500, 92300]
```

Remark: the rule is short enough to memorise. `b = a` *makes a* name*;* `b = a[:]` *makes a* list*. Whenever you are about to modify a list you were given, copy it first unless you were asked to modify it.*

backup = sales[:] -> TWO lists. sales.append(500) left the copy alone.

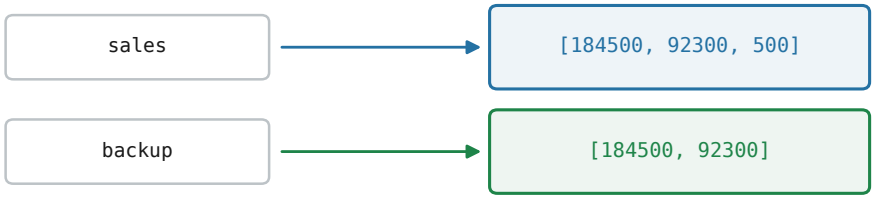


Figure 4. After `backup = sales[:]`: two lists, and `append` reaches only one.

6. Adding and removing

```
sales = [184500, 92300]

sales.append(156800)
sales.insert(0, 73400)
sales.extend([210600, 45900])
print(sales)

last = sales.pop()
first = sales.pop(0)
sales.remove(92300)
print(sales, last, first)
```

Output

```
[73400, 184500, 92300, 156800, 210600, 45900]
[184500, 156800, 210600] 45900 73400
```

❑ **append against extend** – `append` adds **one** item, whatever it is. Given a list, it adds that list as a single item and you get a list inside a list. `extend` adds the items of the other list one by one.

```
a = [1, 2]
b = [1, 2]
a.append([3, 4])
b.extend([3, 4])
print(a, len(a))
print(b, len(b))
```

Output

```
[1, 2, [3, 4]] 3
[1, 2, 3, 4] 4
```

*Pitfall: `.remove(x)` deletes the **first** item equal to `x`, and raises a `ValueError` if there is none. `.pop()` on an empty list raises an `IndexError`. Both are the kind of error that appears only on the unusual input — the empty branch, the missing client.*

7. `sort()` and `sorted()`

❑ **Two functions, one letter apart** – This catches everyone once.

```
sales = [184500, 92300, 156800]

print(sorted(sales))
print(sales)

sales.sort()
print(sales)

print(sales.sort())
```

Output

```
[92300, 156800, 184500]
[184500, 92300, 156800]
[92300, 156800, 184500]
None
```

❑ **Descending, and sorting text** – Both take `reverse=True`. Strings sort alphabetically — but capitals come before all small letters, because the order is by character code, not by dictionary.

```
sales = [184500, 92300, 156800]
print(sorted(sales, reverse=True))

print(sorted(["Oran", "alger", "Medea"]))
print(sorted(["Oran", "alger", "Medea"], key=str.lower))
```

Output

```
[184500, 156800, 92300]
['Medea', 'Oran', 'alger']
['alger', 'Medea', 'Oran']
```

Pitfall: the second result is not a bug. `"O"` and `"M"` are capitals and sort before `"a"`. Any list of names typed by hand will do this. `key=str.lower` is the fix.

Remark: the same `reverse=True` appears on `.sort()` and `.reverse()` does something different: `.reverse()` flips the current order without sorting it.

8. Summarising a list

❑ **Four builtins** – They work on any list of numbers, of any length. This is where a list repays itself: six branches or six hundred, the code is the same.

```
sales = [184500, 92300, 156800, 73400, 210600, 45900]

print(len(sales))
print(sum(sales))
print(min(sales), max(sales))
print(round(sum(sales) / len(sales), 2))
```

Output

```
6
763500
45900 210600
127250.0
```

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i = \texttt{sum(x) / len(x)}$$

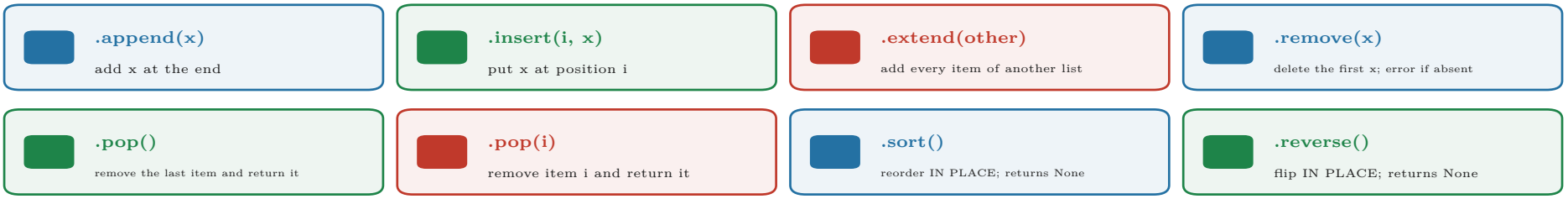


Figure 5. The eight list methods. Note the three marked IN PLACE: they change the list and return `None`.



Figure 6. Read the third line twice. It is the most common list bug there is.

where x_i is the i-th item of the list, n is `len(x)` and $\bar{x}$ is the mean. The formula and the Python are the same statement; `sum` is the $\sum$ and `len` is the n.

Pitfall: `sum([])` is 0, but `len([])` is also 0, so the mean of an empty list is a `ZeroDivisionError`. An empty list is the input that breaks an average, and in real data it arrives eventually — a branch that reported nothing this month.

❑ **`range` and `list`** – `range(a, b)` produces the whole numbers from `a` up to but not including `b`. It is not a list until you make it one.

```
print(list(range(5)))
print(list(range(1, 6)))
print(list(range(0, 50, 10)))
print(sum(range(1, 101)))
```

Output

```
[0, 1, 2, 3, 4]
[1, 2, 3, 4, 5]
[0, 10, 20, 30, 40]
5050
```

9. A list inside a list

❑ **A table** – Each item of the outer list is itself a list — one row. Two indices then reach one cell: the first picks the row, the second the column.

```
rows = [["Medea", 184500],
        ["Alger", 92300],
        ["Oran", 156800]]

print(rows[0])
print(rows[0][0], rows[0][1])
print(len(rows), len(rows[0]))
```

Output

```
['Medea', 184500]
Medea 184500
3 2
```

Remark: this is a table with 3 rows and 2 columns, and it is how data looks before pandas. Chapter 10 replaces it with something far better, but the shape — rows of columns — stays the same.

10. List against string

	`str`	`list`
Written	`"Medea"`	`[184500, 92300]`
Holds	characters only	anything, even mixed
Indexing	identical: from 0, negatives from the end, half-open slices	
Changeable	**no** — methods return a new string	**yes** — methods change it in place
`b = a`	harmless	two names, one list
To copy	not needed	`a[:]`, `list(a)`, `a.copy()`

Quick reference

Making and reading	
`[a, b, c]` `[]`	a list, an empty list
`x[0]` `x[-1]`	first, last
`x[a:b]`	a *list* of the items `a` to `b`−1
`x[::-1]`	a reversed copy
`len(x)`	how many items
Changing (all in place)	
`x[1] = v`	replace one item
`x.append(v)`	add *one* item at the end
`x.extend(y)`	add every item of `y`
`x.insert(i, v)`	put `v` at position `i`
`x.pop()` `x.pop(i)`	remove and return last / item `i`
`x.remove(v)`	delete the first `v`; `ValueError` if absent
Copying — the trap	
`b = a`	**one** list, two names
`b = a[:]`	**two** lists. Also `list(a)`, `a.copy()`
Ordering	
`sorted(x)`	a new sorted list; `x` untouched
`x.sort()`	sorts `x` itself, returns `None`
`reverse=True`	descending, on either
`key=str.lower`	sort text ignoring capitals
Summarising	
`sum(x)` `min(x)` `max(x)`	total, smallest, largest
`sum(x) / len(x)`	the mean. `ZeroDivisionError` if empty
`list(range(a, b))`	the whole numbers `a` to `b`−1

What you can do after this chapter

- Make a list, read one item, and take a slice.
- Say why a list is mutable and a string is not, and what follows from it.
- Recognise the aliasing trap and make a real copy.
- Add and remove items, and tell `append` from `extend`.
- Choose between `sort()` and `sorted()`, and explain why `x = y.sort()` gives `None`.
- Summarise a list with `len`, `sum`, `min`, `max`, and compute a mean safely.
- Read a cell out of a list of lists.

❑ **Next** – Chapter 06 — two more containers: tuples, which cannot be changed, and sets, which cannot repeat.

↪ the list type: docs.python.org/3/tutorial/datastructures.html

↪ sorting: docs.python.org/3/howto/sorting.html